

SQL Queries:

```
DROP PROCEDURE IF EXISTS sp_GetEmployeesByDepartment;  
DROP PROCEDURE IF EXISTS sp_GetEmployeeCountByDepartment;  
DROP FUNCTION IF EXISTS fn_CalculateAnnualSalary;  
DROP TABLE IF EXISTS Orders;  
DROP TABLE IF EXISTS Employees;  
DROP TABLE IF EXISTS Departments;
```

```
CREATE TABLE Departments (  
    DepartmentID INT PRIMARY KEY,  
    DepartmentName VARCHAR(100)  
);
```

```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY,  
    FirstName VARCHAR(50),  
    LastName VARCHAR(50),  
    DepartmentID INT,  
    Salary DECIMAL(10,2),  
    JoinDate DATE,  
    FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)  
);
```

```
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY,  
    CustomerID INT,  
    OrderDate DATE,
```

```
TotalAmount DECIMAL(10,2)
);
```

```
INSERT INTO Departments VALUES
(1, 'IT'),
(2, 'HR'),
(3, 'Finance');
```

```
INSERT INTO Employees VALUES
(101, 'Arjun', 'Kumar', 1, 7200.00, '2022-02-10'),
(102, 'Neha', 'Singh', 1, 9100.00, '2020-08-15'),
(103, 'Vikram', 'Mehta', 2, 5800.00, '2023-01-05'),
(104, 'Ananya', 'Gupta', 2, 5800.00, '2021-06-18'),
(105, 'Kiran', 'Joshi', 3, 10200.00, '2019-09-12');
```

```
INSERT INTO Orders VALUES
(1, 601, '2026-04-10', 540.75),
(2, 602, '2026-05-18', 1299.99),
(3, 603, '2026-06-05', 875.50);
```

```
DELIMITER $$
```

```
CREATE PROCEDURE sp_GetEmployeesByDepartment(IN p_DepartmentID INT)
BEGIN
    SELECT EmployeeID, FirstName, LastName, Salary
    FROM Employees
    WHERE DepartmentID = p_DepartmentID;
END$$
```

```
CREATE PROCEDURE sp_GetEmployeeCountByDepartment(IN p_DepartmentID  
INT)
```

```
BEGIN
```

```
    SELECT COUNT(*) AS TotalEmployees
```

```
    FROM Employees
```

```
    WHERE DepartmentID = p_DepartmentID;
```

```
END$$
```

```
CREATE FUNCTION fn_CalculateAnnualSalary(p_Salary DECIMAL(10,2))
```

```
RETURNS DECIMAL(10,2)
```

```
DETERMINISTIC
```

```
BEGIN
```

```
    RETURN (p_Salary * 12);
```

```
END$$
```

```
DELIMITER ;
```

```
CREATE INDEX IX_Orders_CustomerID_OrderDate
```

```
ON Orders(CustomerID, OrderDate);
```

```
SELECT '----- EXERCISE 1: WINDOW FUNCTIONS (ROW_NUMBER &  
DENSE_RANK) -----' AS 'Execution Log';
```

```
SELECT
```

```
    EmployeeID,
```

```
    FirstName,
```

```
    DepartmentID,
```

```
    Salary,
```

```
    ROW_NUMBER() OVER (PARTITION BY DepartmentID ORDER BY Salary  
DESC) AS SalaryRowNumber,
```

```
DENSE_RANK() OVER (PARTITION BY DepartmentID ORDER BY Salary DESC)
AS SalaryDenseRank
FROM Employees;
```

```
SELECT '----- EXERCISE 2: CALL STORED PROCEDURE -----' AS 'Execution Log';
```

```
CALL sp_GetEmployeesByDepartment(1);
```

```
SELECT '----- EXERCISE 3: EMPLOYEE COUNT -----' AS 'Execution Log';
```

```
CALL sp_GetEmployeeCountByDepartment(2);
```

```
SELECT '----- EXERCISE 4: COMPOSITE INDEX EXECUTION PLAN -----' AS
'Execution Log';
```

```
EXPLAIN SELECT *
FROM Orders
WHERE CustomerID = 601
AND OrderDate = '2026-04-10';
```

```
SELECT '----- EXERCISE 5: EXECUTE SCALAR FUNCTION -----' AS 'Execution
Log';
```

```
SELECT
    EmployeeID,
    FirstName,
    Salary AS MonthlySalary,
    fn_CalculateAnnualSalary(Salary) AS ComputedAnnualSalary
FROM Employees
WHERE EmployeeID = 101;
```

OUTPUT’S:

EmployeeID	FirstName	DepartmentID	Salary	SalaryRowNumber	SalaryDenseRank
102	Neha	1	9100	1	1
101	Arjun	1	7200	2	2
103	Vikram	2	5800	1	1
104	Ananya	2	5800	2	1
105	Kiran	3	10200	1	1

EmployeeID	FirstName	LastName	Salary
101	Arjun	Kumar	7200
102	Neha	Singh	9100
TotalEmployees			
2			

id	parent	notused	detail
3	0	0	SEARCH Orders USING INDEX IX_Orders_CustomerID_OrderDate (CustomerId=? AND OrderDate=?)
EmployeeID	FirstName	MonthlySalary	ComputedAnnualSalary
101	Arjun	7200	86400